FACULDADE DE TECNOLOGIA DE RIBEIRÃO PRETO — FATEC

Relatório Técnico: Projeto Prático de Estrutura de Dados

Tema A — Gerenciamento de Equipamentos de Rede com Árvore Binária de Busca (BST) e Pilha Dinâmica

Aluno:	Christian de Lima	**Curso / Semestre:**	ADS — 3º Semestre
Disciplina:	Estrutura de Dados	**Docente:**	Prof. Me. Heber Castro
Linguagem / Padrão:	C (ISO C11)	**Ambiente / Compilador:**	macOS / Apple clang (gcc)
Repositório Git:	https://github.com/mrdelima/projeto-dados	**Tema Selecionado:**	Tema A — Equipamentos de rede

1. Introdução e Objetivo do Sistema

Este relatório técnico documenta a modelagem, implementação, validação e análise de complexidade do Projeto Prático da disciplina de Estrutura de Dados, desenvolvido em linguagem C (padrão ISO C11) para execução em interface de linha de comando (terminal).

Com base no Tema A (Equipamentos de Rede), o software tem como objetivo gerenciar o cadastro, consulta, listagem ordenada e remoção de ativos de infraestrutura de redes utilizando uma Árvore Binária de Busca (Binary Search Tree — BST) indexada por identificador numérico único (ID). Adicionalmente, o sistema mantém uma estrutura linear auxiliar — uma Pilha Dinâmica (LIFO) — operando de forma totalmente independente da árvore para armazenamento de identificadores inteiros positivos.

2. Arquitetura e Modularização do Projeto

O código-fonte foi organizado de forma modular, separando interfaces (arquivos de cabeçalho em include/) de suas respectivas implementações (arquivos-fonte em src/), além de contar com uma suíte automatizada de testes (tests/) e documentação técnica (docs/). Essa divisão garante baixo acoplamento, facilidade de manutenção e clareza na responsabilidade de cada componente.

```
projeto-dados/
├── include/
│   ├── bst.h          # Definição de Equipamento, NoBST e protótipos das operações da
árvore
│   ├── entrada.h      # Protótipos de leitura segura de linha e validação de entradas
│   └── estrutura.h    # Definição de NoPilha, Pilha e protótipos das operações lineares
├── src/
│   ├── main.c         # Loop principal, apresentação do menu interativo e controle de fluxo
│   ├── bst.c          # Inserção, busca, remoção (3 casos), percurso em-ordem e liberação
│   ├── entrada.c      # Sanitização de buffer, trim de espaços e validação numérica/textual
│   └── estrutura.c    # Operações push, pop, exibição topo-base e liberação da pilha
├── tests/
│   ├── executar_testes.sh   # Script automatizado de compilação, execução e verificação de
leaks
│   ├── resultados.md        # Relatório tabular gerado automaticamente
(entrada/esperado/obtido)
│   └── saida/               # Arquivos individuais de entrada (*.entrada.txt) e saída
(*.txt)
├── docs/
```

```
│   └── relatorio_tecnico.pdf
└── README.md
```

3. Modelagem das Estruturas de Dados

3.1. Registro de Domínio: Equipamento de Rede

Cada ativo de rede gerenciado pela BST é representado pela estrutura Equipamento, dimensionada para armazenar um identificador inteiro positivo e três campos descritivos de tamanho fixo (char[80]), comportando até 79 caracteres úteis mais o terminador nulo ('\0').

```
typedef struct {
    int id;                 // Chave primária única (inteiro > 0 dentro do limite de int)
    char nome[80];          // Nome ou hostname do equipamento (1 a 79 caracteres úteis)
    char tipo[80];          // Categoria do ativo (ex.: Roteador, Switch, Firewall, AP)
    char localizacao[80];   // Local físico ou rack de instalação (1 a 79 caracteres úteis)
} Equipamento;
```

3.2. Árvore Binária de Busca (BST)

A árvore binária de busca organiza os registros dinamicamente na memória heap por meio de nós (NoBST) alocados sob demanda via malloc. A propriedade fundamental da BST é mantida em todas as operações: para qualquer nó N, todos os nós da subárvore esquerda possuem IDs estritamente menores que N.dados.id, e todos os nós da subárvore direita possuem IDs estritamente maiores.

```
typedef struct NoBST {
    Equipamento dados;
    struct NoBST *esquerda;
    struct NoBST *direita;
} NoBST;
```

3.3. Estrutura Linear: Pilha Dinâmica

Para atender ao requisito da estrutura linear auxiliar, optou-se por uma pilha encadeada dinâmica (política LIFO — Last-In, First-Out). Cada nó armazena apenas um valor inteiro positivo e um ponteiro para o elemento imediatamente abaixo, mantendo um descritor com ponteiro direto para o topo.

```
typedef struct NoPilha {
    int id;
    struct NoPilha *proximo;
} NoPilha;

typedef struct {
    NoPilha *topo;
} Pilha;
```

> **Independência entre Estruturas**
>
> A pilha dinâmica opera de maneira completamente desacoplada da árvore binária de busca: permite a inserção de IDs duplicados e aceita identificadores que não estejam previamente cadastrados na BST, funcionando como um histórico/buffer linear independente.

4. Implementação, Operações e Regras de Validação

4.1. Operações da Árvore Binária de Busca (bst.c)

- **Inserção (inserir):** Percorre a árvore comparando o novo ID com os nós existentes. Caso o ID já exista, a inserção é rejeitada para preservar a unicidade da chave. Caso contrário, aloca um novo NoBST na posição folha correspondente.
- **Busca por ID (buscar):** Realiza pesquisa binária recursiva/iterativa descendo à esquerda ou à direita conforme o valor da chave, retornando o ponteiro para o nó encontrado ou NULL.
- **Remoção (remover):** Trata integralmente os três casos clássicos de remoção em BST: (1) Nó folha — libera o nó e ajusta o ponteiro do pai para NULL; (2) Nó com um único filho — substitui o nó removido pelo seu filho direto; (3) Nó com dois filhos — localiza o sucessor em-ordem (menor nó da subárvore direita), copia seus dados para o nó atual e remove recursivamente o nó sucessor.
- **Listagem Em-Ordem (listar_em_ordem):** Executa o percurso in-order (esquerda → raiz → direita), exibindo todos os equipamentos cadastrados em ordem estritamente crescente de ID.
- **Liberação de Memória (liberar_bst):** Executa percurso em pós-ordem (esquerda → direita → raiz) no encerramento do programa para desalocar todos os nós com free.

4.2. Operações da Pilha Dinâmica (estrutura.c)

- **Empilhar (push):** Aloca dinamicamente um novo NoPilha, aponta seu campo proximo para o topo atual e atualiza o topo da pilha em tempo constante.
- **Desempilhar (pop):** Verifica se a pilha está vazia (exibindo aviso amigável sem interromper o programa). Caso haja elementos, remove o nó do topo, exibe o ID removido e libera a memória.
- **Exibição (exibir_pilha):** Percorre os nós encadeados do topo até a base, imprimindo a sequência atual armazenada.

4.3. Sanitização e Validação Rigorosa de Entradas (entrada.c)

Para evitar vulnerabilidades clássicas de entrada em C (como estouro de buffer com scanf ou lixo remanescente no stdin), toda leitura é realizada por linha com controle de limite:

- **Controle de Tamanho de Linha:** Linhas digitadas com mais de 255 caracteres são detectadas, rejeitadas com mensagem de erro e o restante do buffer de entrada até o '\n' é descartado.
- **Validação de IDs:** Utiliza conversão segura (strtol) verificando ausência de caracteres residuais. Aceita sinais positivos e zeros à esquerda (ex.: +5 e 007 são convertidos para 5 e 7), mas rejeita zero, números negativos, decimais (3.5), alfanuméricos (12abc), campos vazios e valores fora do intervalo de int (INT_MAX).
- **Validação de Campos Textuais:** Aplica remoção de espaços em branco nas extremidades (trim). Aceita strings de 1 a 79 caracteres com espaços internos, rejeitando entradas vazias ou compostas exclusivamente por espaços.
- **Encerramento Seguro:** O menu interativo (opções 0 a 7) também detecta EOF (Ctrl+D), acionando a rotina de limpeza completa de memória da BST e da Pilha antes de finalizar o processo.

5. Testes Automatizados e Verificação de Memória

A qualidade e a estabilidade do software foram validadas por meio do script tests/executar_testes.sh, que compila o projeto com flags rigorosas de diagnóstico (-std=c11 -Wall -Wextra -g) e executa uma bateria de cenários automatizados redirecionando arquivos de entrada (*.entrada.txt) para a entrada padrão do programa.

Categoria de Teste	Cenários Cobertos	Resultado

Categoria de Teste	Cenários Cobertos	Resultado
BST — Operações Básicas	Cadastro de múltiplos equipamentos, busca por ID existente/inexistente e listagem ordenada (in-order).	**PASS**
BST — Remoção (3 Casos)	Remoção de nó folha, nó com 1 filho (esquerdo e direito), nó com 2 filhos e remoção da raiz.	**PASS**
Validação de IDs e Textos	Rejeição de ID duplicado, zero, negativo, texto, overflow de int, texto vazio, >79 chars e linha >255 chars.	**PASS**
Menu e Fluxo de Controle	Tratamento de opções inválidas no menu e encerramento limpo via opção 0 e via EOF (Ctrl+D).	**PASS**
Pilha Dinâmica e Independência	Push de IDs repetidos e inexistentes na BST, exibição topo-base, pop sequencial e pop em pilha vazia.	**PASS**
Integridade de Memória	Execução monitorada via leaks --atExit (macOS) / valgrind (Linux) após ciclo completo de operações.	**0 leaks (0 bytes)**

Comprovação de Ausência de Vazamentos: A execução do utilitário de diagnóstico de memória confirmou a liberação integral de todos os blocos alocados dinamicamente, retornando exatamente: "0 leaks for 0 total leaked bytes".

6. Análise de Complexidade Assintótica (Big-O)

Abaixo apresenta-se a análise de complexidade de tempo e espaço das operações implementadas, considerando n como o número de elementos armazenados na respectiva estrutura e h como a altura da Árvore Binária de Busca.

Estrutura	Operação	Caso Médio	Pior Caso	Espaço Auxiliar
BST	Inserção (inserir)	**O(log n)**	**O(n)**	O(h) pilha recursiva
BST	Busca por ID (buscar)	**O(log n)**	**O(n)**	O(h) pilha recursiva
BST	Remoção (remover)	**O(log n)**	**O(n)**	O(h) pilha recursiva
BST	Listagem Em-Ordem	**O(n)**	**O(n)**	O(h) pilha recursiva
BST	Liberação Total (free)	**O(n)**	**O(n)**	O(h) pilha recursiva
Pilha	Empilhar (push)	**O(1)**	**O(1)**	O(1) auxiliar
Pilha	Desempilhar (pop)	**O(1)**	**O(1)**	O(1) auxiliar
Pilha	Exibição / Liberação	**O(n)**	**O(n)**	O(1) auxiliar

Discussão: Na Árvore Binária de Busca, quando as chaves são inseridas em ordem aleatória, a altura mantém-se próxima de $h \approx \log_2(n)$, conferindo desempenho logarítmico O(log n) às operações de busca, inserção e remoção. Contudo, como a estrutura não realiza rebalanceamento automático (como ocorre em árvores AVL ou Rubro-Negras), a inserção sequencial de IDs estritamente crescentes ou decrescentes degenera a árvore em uma lista encadeada (h = n), elevando o custo para O(n). Na Pilha Dinâmica, o acesso direto via ponteiro topo garante inserções e remoções em tempo constante O(1), independentemente do volume de itens armazenados. Em termos de memória total, ambas as estruturas ocupam espaço linear O(n) proporcional à quantidade de nós ativos.

7. Conclusão

O desenvolvimento do sistema permitiu aplicar na prática conceitos fundamentais de gerenciamento dinâmico de memória em C, ponteiros, recursividade e estruturas de dados lineares e hierárquicas. A separação modular entre interface, lógica de domínio, sanitização de entrada e automação de testes resultou em uma aplicação robusta, resiliente a entradas malformadas e comprovadamente livre de vazamentos de memória.